

RINS Task 2 Report

Team Name Placeholder

Student One

Student Two

Student Three

1 Introduction

This report covers Task 2 of the RINS 2026 project, which combines perception, navigation and decision logic into one pipeline running across two rooms.

1.1 The Task

The robot starts in the **first room** and drives through it looking for people. When it finds someone, it greets them and asks for a task.

The four tasks. A person hands out one of four jobs: count **rings**, inspect **barrels**, or detect **anomalies** on the red or the green cell. Counting rings means remembering every ring seen in the first room together with its color. Inspecting barrels means remembering every barrel, its color and whether it lies horizontal, then driving closer to any horizontal one to check it for a leak; a leaking barrel is announced out loud. Detecting anomalies means driving to the belt with the matching colored line, raising the camera, and inspecting every tile for defects.

The no go zone. A yellow line in the first room marks a no go zone. The robot must never cross it, which constrains every path it plans there.

The second room. Once the first room is searched and the assigned tasks are done, the robot moves to the entrance of the second room, picks up a blue line, and follows it through its intersections until it reaches the **CTO**, who stands next to the line.

The report. Reaching the CTO triggers the report. It gives the ring count and the per color counts; a table of every barrel with its color, a horizontal flag and a leaking flag, plus a picture of each leaking barrel; and a table of every inspected tile with a defect flag, plus the tile image and its anomaly mask for the defective ones. Each task also records who asked for it.

2 Methods

2.1 Face Detection

YOLO runs the whole image through a single network pass and returns boxes, each with a class, a confidence and image coordinates. That single pass is what lets the robot detect faces while it keeps moving. We use the box only as a pointer: it tells us where the face sits in the image, and we read the matching **depth** to lift that pixel into a 3D position on the map. The position is what the rest of the system needs, not the picture.

2.1.1 Calculating the Face Destination

To start a conversation the robot must stop in front of a face. Our first goal was a point 40 cm ahead of the face along the *robot to face line*. Head on that works, but at a shallow angle it drops the goal beside the face or into the wall: the robot stalls and rarely ends up turned at the person (Figure 1). The quantity that fixes this is the **wall normal**, so the rest of this section computes it and turns it into a goal.

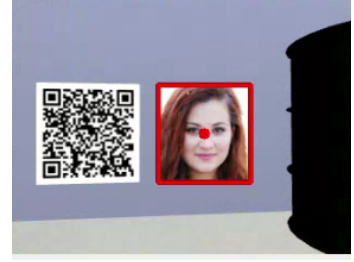


Figure 1: Direct approach fails at a shallow angle.

Wall normal. We tried the lidar first, and it was too fragile near corners and partial occlusions. Instead we read the wall from the **occupancy grid** the robot already keeps for navigation, whose cells are *free*, *occupied* or *unknown* and stay steadier than any single scan.

To pick the right wall we cast a **ray** from the robot toward the face and take the first free to occupied transition as its surface (Figure 2). A breadth first search would be wrong here: if the face sits just past the wall, the nearest occupied cell can be the far side of that same wall, while the ray always hits the side the robot will see.

We fit the 7 nearest wall cells with **RANSAC**, which shrugs off the stray cells at a corner, and get a wall direction $\mathbf{d} = (d_x, d_y)$. The 2D normal is the in plane perpendicular, from a cross product against the vertical axis $\hat{\mathbf{z}}$:

$$\mathbf{n} = \mathbf{d} \times \hat{\mathbf{z}} = (d_y, -d_x, 0).$$

Dropping the zero component leaves $\mathbf{n} = (d_y, -d_x)$, flipped to point out of the wall toward the robot.

Goal. The stopping **goal** pushes the face point out along this normal, facing back into the wall:

$$\mathbf{p}_{\text{goal}} = \mathbf{p}_{\text{face}} + d_{\text{stop}} \hat{\mathbf{n}}, \quad \theta_{\text{goal}} = \text{atan2}(-n_y, -n_x),$$

where $\mathbf{p}_{\text{face}}$ is the face position, $d_{\text{stop}} = 0.4\text{m}$ is the stopping distance, and $\hat{\mathbf{n}}$ is the unit wall normal. The heading θ_{goal} makes the robot arrive looking straight at the person from any approach angle.

2.2 Face Classification

Face classification was used after the robot had already detected and approached a person. The goal of this module was not to estimate the person's position, but to identify which known person was in front of the robot and to retrieve their role and gender information for the later decision logic and conversation.

The method is based on face embeddings. A face embedding is a compact numerical representation of a face image. Images of the same person should produce similar embeddings, while images of different people should be farther apart in the embedding space. In our implementation, a FaceNet-style model, **InceptionResnetV1** pretrained on VGGFace2, is used to compute these embeddings.

For classification, the embedding computed from the live camera image is compared with precomputed embeddings stored in the personnel database. The nearest stored embedding is selected using Euclidean distance. The selected entry gives the person's name, role, and gender.



Figure 2: Ray cast versus BFS to the wall.

2.3 Ring Detection

Text to be added.

2.4 Barrel Detection

2.4.1 Detection

Text to be added.

2.4.2 Computing Inspection Goals

A leak shows up on the **top or bottom face** of a horizontal barrel, so the robot needs a clean view of one of them. Seen from above the barrel is symmetric about both axes, and we know its orientation only up to that symmetry. That is fine: it leaves four equally good viewing spots, one off each corner, each looking straight at a top or bottom face (Figure 3).

Four goals. We already have the barrel center and orientation from detection, so we build the four goals by shifting the center along the orientation vector and the vector perpendicular to it:

$$\mathbf{p}_{ij} = \mathbf{c} + i a \hat{\mathbf{u}} + j b \hat{\mathbf{v}}, \quad i, j \in \{+1, -1\},$$

with $\mathbf{c}$ the center, $\hat{\mathbf{u}}$ the unit orientation along the barrel axis, $\hat{\mathbf{v}}$ the unit perpendicular, and a, b the offsets along and across. At each goal the robot faces the center with an adjustable offset φ so the camera lines up with a top or bottom face:

$$\theta_{ij} = \text{atan2}(c_y - p_{ij,y}, c_x - p_{ij,x}) + \varphi.$$

Pruning. Some goals land inside a wall or too close to one to be useful. For each goal we measure the distance to the nearest wall and drop it if that distance is too small or the goal sits in a wall. The robot visits whatever survives.

2.5 Anomaly Detection

SuperSimpleNet gives us both an image level anomaly score and a pixel level anomaly map: the score decides whether a tile is defective, the map shows where. We trained it fully supervised, since the dataset shipped with anomaly masks, so the model learned the defect location directly, which the report needs as a mask.

The first model was trained on the raw dataset. In Gazebo the simulated lighting made the ceramic tiles much brighter than the training images, a **domain shift** that hurt detection. We brightened the tile images into an augmented dataset, retrained from the existing model, and the augmented model held up under the simulator’s lighting. That is the model the node runs.

2.5.1 Navigating Along the Belt

Along the belt we publish velocities straight to `/cmd_vel` and skip `nav2`. This close to the wall, localization drifts, so a map pose would be unreliable; instead we read the belt straight from the **lidar**, which stays accurate at short range.

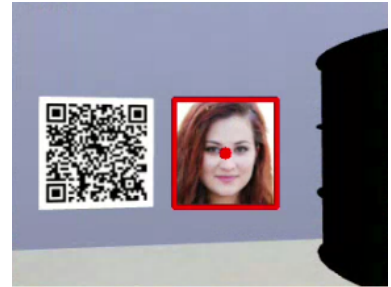


Figure 3: Inspection goals around a horizontal barrel.

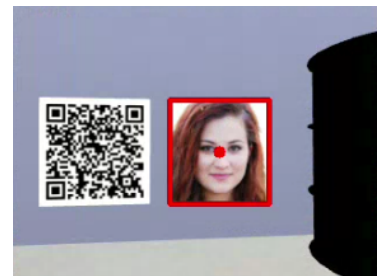


Figure 4: Line fit to the left

Line fit. nav2 brings the robot to a fixed keypoint at the head of the belt. From there we fit a line with **RANSAC** to the 90 degree scan cone on the robot’s left, since the belt runs along that side (Figure 4). The fit hands back the perpendicular distance to the belt and the belt’s orientation in the robot frame, which we use to set the exact distance from the belt and to turn precisely perpendicular before the run.

PD wall following. We poll the distance every controller tick and steer with a PD law:

$$e = d_{\text{true}} - d_{\text{target}}, \quad \omega = K_P e + K_D \frac{e - e_{\text{prev}}}{\Delta t},$$

where d_{true} is the measured perpendicular distance, d_{target} the distance we want to hold, e_{prev} the previous tick’s error, Δt the tick period, and K_P, K_D the gains. When the robot drifts away from the belt d_{true} grows past d_{target} , the error turns positive, and ω steers it back toward the belt; drift too close and the error goes negative and pushes it out again. The D term damps the correction, so the robot holds a straight line instead of weaving. We call the anomaly service at fixed distance steps as we move down the belt.

2.6 Follow Line

The top camera is raised to look down at the floor just in front of the robot. The robot drives along the line, and at a crossroad it always turns right. When the line ends or an obstacle blocks the way ahead, it turns around and drives back.

Keying the line. We key the **blue** of the line on two rows of the image, one close to the robot and one further ahead (Figure 5, where the keyed line is drawn in black). To make the mask robust we apply a *closing*, a dilation followed by an erosion, which fills small gaps and leaves one solid segment per row.



Figure 5: Two sample rows; the line mask is drawn in black.

Steering with PD. We steer with a PD law whose terms come from the image:

$$e_P = c - x_{\text{near}}, \quad e_D = x_{\text{near}}^R - x_{\text{far}}^R, \quad \omega = K_P e_P + K_D e_D,$$

where c is the image center column, x_{near} is the center of the followed segment on the near row, and $x_{\text{near}}^R, x_{\text{far}}^R$ are the right edges of that segment on the near and far rows. The idea matches the belt controller, so we do not repeat it.

Why the right edge. With the P term alone the robot jumps a little at every crossroad and never drives smoothly, which is why we added the D term. But the D term has to be read carefully. At a crossroad the line first thickens and then splits in two. Measured from the segment centers, the center jumps the moment the front line splits from one thick blob into two, which is exactly the jump we were removing. The **right edge** does not move when the line splits (Figure 6), so tracking it keeps the D term smooth.

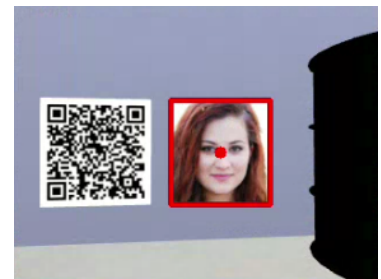


Figure 6: Right edge holds as the line splits (line in black).

Junctions from the top. Approaching a junction from the top, the other arm shows up on the robot's right (Figure 7). Since the robot follows the rightmost line, it would snap into a sharp right turn. We reject that arm by requiring a newly detected segment to stay within n pixels of the segment we are already following.

Obstacles ahead. We also watch the lidar like a **sonar**. We split the 90 degree cone in front of the robot into 7 bins and average the range in each. If any bin reads too close, the robot turns around.

2.7 Navigation Around Map and Yellow Line

The robot moves around the first room through a predefined set of poses, each with a position and an orientation, driven by nav2. Getting it smooth took some tuning, so it is worth a quick look at how the nav2 controller actually drives.

Carrots and candidate paths. The controller samples several candidate trajectories on each cycle, seeded from the previous path so it never scores complete nonsense, and the cost function picks the best one. The robot does not track that path exactly, since it jitters slightly between samples. Instead it chases a **carrot** placed on the path at a fixed lookahead distance (Figure 8), which keeps the motion smooth.

Inflation and braking. The robot kept braking hard near the inflated walls in the costmap, and worse, if it strayed onto the inflated area by mistake it refused to drive forward and got stuck. Our localization was accurate, so we turned off the nav2 collision checking and shrank the inflation a lot, which removed both problems without real risk.

Carrot distance. The lookahead was also too long. In tight spaces the carrot pulled the robot over the inflated edge instead of hugging the path, which was the other reason it got stuck in corners. We made the lookahead **scale with speed**: slow means follow the path almost exactly, fast means more room to maneuver. The shorter carrot at low speed cleared the tight corners.

Keep out zones. The yellow no go zone needed a real mechanism. We built a second map, shaped like the navigation map, that marks everything inside the yellow lines as a **keep out zone**, and fed it into the cost function. Every planned path then routes around the keep out areas on its own.

2.8 Decision Logic

For the first task a small **state machine** was enough, with three states: **FollowPath**, **VisitSubject** and **GreetSubject**. The second task is far more tangled, and a flat state machine stops being readable, so we moved to **py_trees_ros** and built a **behaviour tree**.

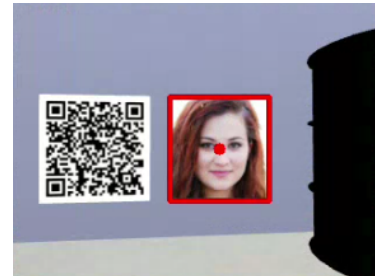


Figure 7: Approaching a junction from the top (line in black).

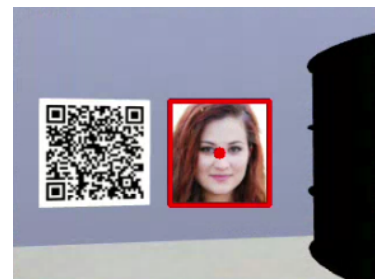


Figure 8: Carrot on the planned path.

Behaviours. A behaviour is a leaf of the tree. On every tick (10 Hz) it does a slice of work and returns one of three results: *Running* if it needs more ticks, *Success* if it is done, or *Failure* if it cannot finish.

Sequences. A sequence is an inner node holding an ordered list of children. It runs the first child, and only when that child returns *Success* does it move to the next; a *Failure* stops the whole sequence there. A sequence can hold behaviours or other sequences, so the tree is built by nesting sequences of behaviours. A memory flag decides whether a sequence re-checks its children from the start on every tick or resumes where it left off.



Figure 9: Example sequence: visiting a detected face.

Why it helped. This made debugging direct: a failure points at the exact behaviour that caused it. It also made the code reusable, since many sequences share the same “go to position X” behaviours, so we wrote each piece once and tested it everywhere. As an example, the sequence that visits a detected face chains three behaviours: drive to the goal from the wall normal, run the conversation, and mark the chosen task active (Figure 9).

3 Implementation and Integration

3.1 ROS Nodes Architecture

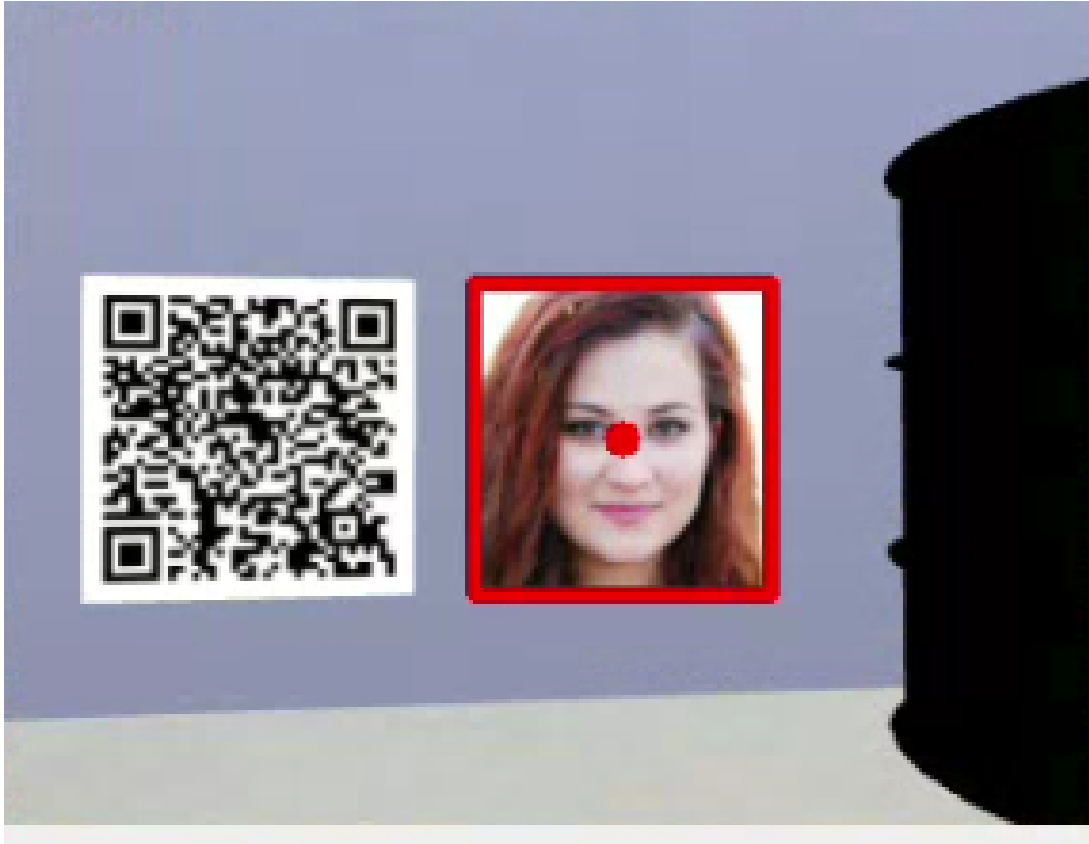


Figure 10: The ROS node and topic graph, from `rqt_graph`.

The **movement** node is the central controller and holds the decision logic of Section 2.8; every other node feeds it or serves it. Figure 10 shows the full graph.

Perception nodes. `detect_faces` publishes faces on `/face_detect`, `detect_rings` publishes rings on `/ring_detect`, the barrel detector publishes `BarrelDetect` messages, and `anomaly_detection` exposes the `detect_anomalies` service.

Geometry services. `map_query` serves `wall_normal_at` for faces and `nearest_wall` for barrel goals, `line_fit` serves `line_fit_in_direction` for the belt, and `blue_line` publishes `BlueLineStatus` on `/blue_line`.

Interaction. `conversation` serves `converse_person`, `classify_face` serves `classify_face`, and `speak` listens on `/speak` and voices whatever is published there.

3.2 Face Detection

`detect_faces` subscribes to the synchronized OAK-D RGB image and depth point cloud, so each pair shares a moment in time. It runs YOLO on the RGB image, takes the center pixel of each box, looks that pixel up in the organized point cloud for a 3D point in the camera frame, then wraps it in a `PoseStamped` and transforms it to the `map` frame with TF2.

To avoid publishing noise, the node holds new detections as candidates and keeps a **running average** of each position. A candidate becomes a real face only after it has been seen past an acceptance threshold, at which point it gets an id. Accepted faces are published on `/face_detect` as `FaceDetect`:

```
float32 x
float32 y
int8 id
```

Visiting and greeting. `movement` subscribes to `/face_detect` and queues each new id. To visit one it calls `wall_normal_at` on `map_query`, passing the face and robot positions; the service returns the wall point and its normal (Section 2.1.1). `movement` turns those into the 40 cm stopping goal, sends it to `nav2`, and on arrival calls `converse_person` on `conversation`. That service takes the person's gender and returns the chosen **task** string, which `movement` marks active.

Re-driving and fallback. If `detect_faces` republishes the same id, the true face has moved, so `movement` recomputes the goal from the new position. If `wall_normal_at` reports no usable wall, `movement` falls back to the direct robot to face approach from Section 2.1.1.

3.3 Face Classification

The face classification module is implemented as the ROS 2 node `classify_face`. It exposes the service `/classify_face`, which is called by the central `movement` module when the robot is close enough to a detected person.

The node subscribes to the OAK-D RGB image topic `/oakd/rgb/preview/image_raw` and keeps the latest frame. When the service is called, it collects several fresh frames, with ten samples used by default. For each frame, the image is converted to OpenCV format and YOLO is run with class 0, corresponding to a person. The first detected bounding box is cropped from the image and used as the input for face embedding.

The crop is passed through `InceptionResnetV1`. The resulting embedding is compared to all entries in `personnel_embeddings_dict.py`. The nearest match is selected using Euclidean distance.

To make the result more stable, the module does not rely on only one frame. It repeats the detection and embedding process over multiple samples and stores a vote for the nearest matching person in each frame. The final identity is chosen by majority vote. If no person is detected in the first frame, or if no valid match is found in any frame, the service returns `success = false`.

On success, the service returns the matched name, role, gender, and a message describing how many samples voted for the selected identity. The `movement` module stores this information in the current pending person and updates the visualization label.

3.4 Ring Detection

`detect_rings` publishes each ring on `/ring_detect` as `RingDetect`, which carries the position, an id and a color. `movement` does little at drive time: it records where each ring was seen and its color, and keeps them for the report.

3.5 Barrel Detection

The barrel detector publishes `BarrelDetect` for every barrel it sees, whether or not a task is active. The message carries the position and normal, an id, the color, a horizontal flag, a leaking flag and a path to a saved image.

Visiting horizontal barrels. Once a person sets the barrel task active, movement visits every horizontal barrel it has stored, and any new one it later sees. For each candidate goal around a barrel (Section 2.4.2) it calls `nearest_wall` on `map_query`, which returns the closest wall point; movement keeps the goal only if that point is far enough away. It then drives to a surviving goal with `nav2`.

Announcing a leak. On arrival the detector re-sends a `BarrelDetect` with the leaking flag set if the barrel is leaking, with no service call needed. When movement sees that flag it publishes to `/speak` and the `speak` node says the warning out loud. The image path is stored for the report.

3.6 Anomaly Detection

When a person sets a red or green anomaly task active, movement drives to a fixed keypoint at the belt with `nav2`, then takes over on `/cmd_vel`: it uses the lidar line fit to set its distance and turn perpendicular (Section 2.5.1), drives down the belt, and calls `detect_anomalies` every fixed step. Each call returns a result and the paths to a saved tile image and mask, which movement stores per tile for the report.

The detector. `anomaly_detection` keeps the latest top camera frame and acts only when its `detect_anomalies` service is called. It first finds the `tile`: grayscale, a fixed intensity threshold, a morphological closing to fill gaps, and a border strip removed so edge objects are ignored. It takes the largest contour, its convex hull, and approximates four corners with `approxPolyDP`. A `homography` then warps that quad to a square, so the model always sees the tile head on, and LAB brightness normalization evens out lighting. The square goes to SuperSimpleNet, and the score against a threshold decides `defected`, `not_defected`, or `not_found` when no tile is found. The response carries the saved tile and mask paths.

3.7 Line Following

`blue_line` does the vision and publishes `BlueLineStatus` on `/blue_line`: a state that is `LOST`, `LINE` or `CROSSROAD`, with the line's left and right offsets. movement turns the offsets into the P and D terms of Section 2.6, turns right on `CROSSROAD`, and turns around on `LOST` or when the obstacle bins read too close.

Finding the CTO. The one step specific to this task is the stop condition. When the line is lost or an obstacle blocks the way, movement calls `classify_face` on `classify_face`, which grabs a live frame and returns the person's name, role and gender. If the role is the CTO, the robot has arrived and report generation fires; otherwise it keeps following the line.

3.8 Speech and Remaining Services

`speak` is a shared utility: it listens on `/speak` and voices whatever is published there, which the greeting, the leak alert and the task prompts all use. The `line_fit` node's `line_fit_in_direction` service, used for the belt, fits a line to a lidar cone in a given direction and returns the perpendicular distance and the rotation needed to face it square on.

4 Results

Text to be added.

5 Division of Work

5.1 Who Did What

Text to be added.

5.2 Percentages

Text to be added.

6 Conclusion

Text to be added.